

The results are identical to the test code from [Listing 4-3](#). However, there is one highly significant difference between the two test programs: now there is only the single global variable `accountsList`. Each account has a unique account number, which we use to access a specific account. We have taken an important step in reducing the number of global variables.

Another important thing to note here is that we made some fairly major changes to the main program, but we did not touch anything in the `Account` class file. OOP often allows you to hide details at different levels. If we assume that the code of the `Account` class takes care of details related to an individual account, we can concentrate on ways to make the main code better.

Notice also that we're using the variable `oAccount` as a *temporary* variable. That is, whenever we create a new `Account` object, we are assigning the result to the variable `oAccount`. Right after doing that, we append `oAccount` to our list of accounts. We never use the variable `oAccount` in calls to any method of a specific `Account` object. That way, we can reuse the variable `oAccount` to receive the value of the next account that is created.

Multiple Objects with Unique Identifiers

The `Account` objects must be individually identifiable so each user can make deposits and withdrawals and get the balance of their specific account. Using a list for our bank accounts works, but there's a serious flaw. Imagine we have five accounts, numbered 0, 1, 2, 3, and 4. If the person who owns account 2 decides to close their account, we'd likely use a standard `pop()` operation on the list to delete account 2. This would cause a domino effect: the account that was in position 3 is now in position 2, and the account that was in position 4 is now in position 3. However, the users of these accounts still have their original account numbers, 3 and 4. As a result, the customer who owns account 3 will now get the information for the previous account 4, and account number 4 is now an invalid index.

To handle large numbers of objects with unique identifiers, we generally use a dictionary. Unlike a list, a dictionary will allow us to delete accounts without altering the account numbers associated with them. We build each key/value pair with an account number as the key and an `Account` object as